

Projeto 1 — Benchmark de Suavização/Desfoque Gaussiano Interativo: Implementação paralela em memória distribuída com MPI. (Etapa 2)

Relatório acadêmico — DEC107 — Processamento Paralelo

28 de maio de 2026

Projeto 1 — Benchmark de Suavização/Desfoque Gaussiano Interativo: Implementação paralela em memória distribuída com MPI. (Etapa 2)

Autores: Arthur de Oliveira Moreira e João Vitor Guimarães dos Santos

Disciplina: DEC107 — Processamento Paralelo

Data: 01 de Junho de 2026

1. Introdução

Descrição do objetivo do projeto com foco na Etapa 2 - MPI:

O objetivo principal deste projeto, concentrado em sua segunda etapa, consiste em projetar, implementar e avaliar o desempenho de uma versão paralela do algoritmo de Suavização Gaussiana Interativa utilizando a biblioteca MPI para ambientes de memória distribuída. Busca-se investigar o impacto da decomposição de domínio — isto é, o particionamento da matriz da imagem — e da troca de mensagens entre múltiplos nós de processamento no tempo total de execução da aplicação.

O foco está em realizar um *benchmark* para mensurar a relação de compromisso entre o processamento computacional local de cada nó e o *overhead* de comunicação introduzido pelas barreiras de sincronização, que são obrigatórias a cada iteração do filtro de desfoque. Através do levantamento empírico de métricas de escalabilidade, como *speedup* e eficiência, o trabalho visa determinar o ponto de saturação da rede e a granularidade ideal na divisão dos dados, validando a viabilidade do uso de um cluster para o processamento paralelo desse algoritmo.

Apresentação dos conceitos teóricos relevantes: Suavização/Desfoque Gaussiano, paralelismo de memória distribuída, MPI, decomposição de domínio, troca de halos:

Desfoque Gaussiano é uma técnica de processamento de imagem que reduz ruídos e suaviza detalhes através de uma convolução local. O valor de cada pixel é substituído pela média ponderada de seus vizinhos,

utilizando uma máscara ou kernel (3x3 ou 5x5) que atribui pesos maiores aos pixels centrais. Para diminuir a quantidade de operações de ponto flutuante, o algoritmo aplica uma máscara pequena repetidas vezes de forma iterativa, em vez de calcular uma máscara grande em um único passo. O desafio dessa abordagem é a dependência gerada, já que o próximo ciclo de desfoque só pode iniciar após a matriz inteira ter sido atualizada.

Como a carga de cálculo cresce muito rápido com o tamanho da imagem e o número de iterações, a aplicação precisa ser paralelizada. O modelo adotado é o de memória distribuída, gerenciado via MPI. Através da decomposição de domínio, a matriz da imagem é fatiada horizontalmente e distribuída entre os nós do cluster. Como não existe memória compartilhada entre eles, cada processo roda de forma isolada, resolvendo apenas a sua parte da imagem.

O problema geométrico dessa divisão aparece nos limites de cada bloco. Para calcular a convolução dos pixels que ficam nas bordas de uma fatia, o processo precisa ler a linha vizinha, que agora pertence fisicamente a outro nó. Para consertar essa "quebra" na matriz, usamos a troca de halos (*halo exchange*). Antes de iniciar a matemática de cada iteração, os processos enviam e recebem mensagens via MPI com as linhas de contorno, preenchendo espaços extras alocados nas extremidades das matrizes (células fantasmas). Essa sobreposição temporária garante que o cálculo da borda funcione perfeitamente, mesmo com os dados fragmentados na rede.

A Importância da Análise de Desempenho em Sistemas de Memória Distribuída:

Em implementações de memória distribuída, a análise de desempenho é necessária para comprovar se a paralelização realmente entrega vantagens práticas. Diferente de sistemas de memória compartilhada, a troca de mensagens entre os nós da rede gera um *overhead* inerente à comunicação. No caso do desfoque gaussiano iterativo, é preciso medir rigorosamente se o tempo gasto para particionar e transferir as matrizes da imagem entre os processos não acaba anulando o ganho de tempo obtido pela divisão do cálculo.

Além disso, o levantamento de métricas como *speedup* e eficiência é o que permite identificar gargalos físicos ou lógicos no sistema. Como o algoritmo exige barreiras de sincronização a cada iteração do filtro, qualquer desbalanceamento de carga faz com que processos fiquem ociosos esperando os demais. A análise empírica de desempenho serve justamente para ajustar a granularidade dos dados enviados e garantir um uso racional do hardware, escalando o processamento no cluster sem saturar a rede.

2. Metodologia

Descrição da implementação paralela com MPI, detalhando quais funções MPI foram usadas, como foi feita a decomposição de domínio e a troca de halos:

Dado que o padrão MPI opera sob o paradigma de memória distribuída, onde cada processo possui um espaço de endereçamento isolado e não compartilha variáveis, a paralelização da imagem exigiu uma reformulação completa da estrutura de dados e dos mecanismos de sincronização temporal.

A. Decomposição de Domínio Dinâmica e Balanceamento de Carga

A abordagem empregada para a divisão do trabalho computacional foi a **decomposição de domínio por faixas horizontais** (linhas). Esta escolha é ideal para matrizes em linguagem C, cujo armazenamento em RAM ocorre de forma contígua linha por linha.

Para garantir a robustez e evitar falhas quando a altura da imagem (H) não é perfeitamente divisível pelo número de processos (P), implementou-se uma lógica de balanceamento dinâmico de carga:

- Cada processo calcula individualmente sua quantidade de linhas locais através da expressão:

$$local_rows = \frac{H}{P} + (rank < (H \bmod(P)) ? 1 : 0)$$

- As linhas excedentes ($H \bmod P$) são distribuídas gradativamente entre os primeiros nós do cluster (um elemento adicional por processo), mantendo o ecossistema balanceado e livre de ociosidade excessiva.
- No processo raiz (Rank 0), são calculados dois vetores auxiliares dinâmicos: `sendcounts` (armazenando a quantidade total de pixels de cada partição, calculada por `local_rows X cols`) e `displs` (mapeando o deslocamento linear ou índice de início de cada bloco no vetor global).
- Com este mapeamento concluído, o Rank 0 invoca a primitiva de comunicação coletiva `MPI_Scatterv`, que distribui os pedaços exatos da imagem original diretamente para um buffer local sequencial (`local_current`) alocado por todos os processos.

B. Estrutura de Memória Local e Troca de Halos (Halo Exchange)

Para aplicar o filtro espacial sobre um pixel, o algoritmo de convolução precisa acessar os seus vizinhos adjacentes dentro de uma vizinhança ditada pelo raio do kernel. Enquanto pixels centrais possuem vizinhos locais disponíveis, os pixels situados nas linhas limítrofes (fronteiras da fatia) dependem de dados que residem na memória RAM de nós vizinhos.

Para solucionar essa dependência sem violar o isolamento de memória distribuída, expandiu-se verticalmente o buffer de trabalho de cada processo criando **Zonas de Halo** (*ghost cells*). Alocou-se um espaço contíguo unidimensional denominado `local_current_with_halos` cujo tamanho total compreende $(local_rows + 2 \times pad) \times cols$ elementos. O mapa de memória deste buffer divide-se em:

1. **Halo Superior:** Ocupa as linhas de 0 a `pad - 1`, destinando-se a receber as linhas de borda real enviadas pelo processo imediatamente acima (`rank - 1`).

2. **Dados Reais Locais:** Ocupa as linhas de pad até pad + local_rows - 1, onde os pixels recebidos via MPI_Scatterv são copiados a partir do deslocamento de memória inicial pad X cols.
3. **Halo Inferior:** Ocupa as linhas de pad + local_rows até pad + local_rows + pad - 1, destinando-se a receber os dados vindos do processo imediatamente abaixo (rank + 1).

Como o desfoque gaussiano opera de forma estritamente iterativa (1000 passadas), os valores de fronteira alteram-se a cada iteração, tornando os dados recebidos inicialmente obsoletos (*stale data*). Assim, determinou-se como requisito temporal obrigatório que a **Troca de Halos** ocorra dentro do laço principal, imediatamente antes da computação local.

A sincronização de fronteiras é efetuada por meio da primitiva MPI_Sendrecv, eliminando riscos de bloqueios mútuos (*deadlocks*) ao realizar o envio e recebimento simultâneo de mensagens:

- **Sincronização do Halo Superior:** O processo envia suas primeiras pad linhas de dados reais para o vizinho de cima (target_top = rank - 1) e recebe as linhas de halo inferior do vizinho de baixo no buffer de destino.
- **Sincronização do Halo Inferior:** O processo envia suas últimas pad linhas de dados reais para o vizinho de baixo (target_bottom = rank + 1) e recebe as linhas de halo superior vindas do processo de cima no buffer de destino.
- **Tratamento de Bordas Físicas:** Para as extremidades reais da imagem, onde não existem processos adjacentes, o MPI define os alvos como MPI_PROC_NULL, o que anula de forma segura as chamadas de rede associadas. Nestes casos, o Rank 0 preenche seu halo superior e o último Rank preenche seu halo inferior manualmente via laços de cópia (memcpy), replicando suas próprias linhas reais limítrofes para dentro das zonas fantasmas, espelhando o comportamento exato do código sequencial.

C. Otimização Algorítmica e Convolução Local

Após a garantia de que o buffer expandido possui todas as vizinhanças verticais atualizadas em rede, a computação da convolução local é disparada através da função customizada `apply_convolution_mpi`. Visando uma medição de desempenho justa e de alta eficiência, aplicou-se a técnica de *hoisting* de alocação : os buffers dinâmicos auxiliares de trabalho (`local_padded`, `local_current` e `local_next`) foram instanciados inteiramente fora do laço iterativo de 1000 passadas, mitigando o overhead repetitivo de chamadas ao sistema operacional (`malloc/free`).

A função paralela otimizada dispensa verificações condicionais (if) para as bordas verticais da imagem dentro do laço de pixels, uma vez que as zonas fantasmas já contém dados válidos e contíguos trazidos pela comunicação MPI. O código restringe-se a realizar a replicação manual apenas para as bordas receberem tratamento nas colunas laterais (esquerda e direita de cada linha). Em seguida, mapeia o cálculo aritmético

no buffer temporário `local_padded` de dimensões $(\text{local_rows} + 2 \times \text{pad}) \times (\text{cols} + 2 \times \text{pad})$, gravando os resultados no vetor de saída `local_next`. Ao término da iteração, os ponteiros de estado são alternados para preparar o passo seguinte.

D. Reconstrução da Imagem com MPI_Gatherv

Concluídas as 1000 iterações, os dados finais processados em cada partição (`local_current`) precisam ser consolidados e ordenados para a geração do arquivo de saída PGM. Devido à heterogeneidade dos tamanhos das fatias provocada pelo desbalanceamento dinâmico em imagens de resoluções ímpares, a primitiva convencional `MPI_Gather` (que exige tamanhos idênticos de envio) tornou-se inviável, sob risco de corrupção ou falhas de segmentação.

Em seu lugar, utilizou-se a primitiva `MPI_Gatherv`, que reconstrói o quebra-cabeça bidimensional de bytes consultando os vetores de controle `sendcounts` e `displs` previamente estruturados no processo mestre, garantindo uma recomposição perfeita e centralizada no ponteiro `output` do Rank 0.

Descrição do hardware/ambiente de execução utilizado nos testes (cluster/nós):

Modelo do Equipamento: Acer Predator Helios Neo 16.

Processador (CPU): Intel® Core™ i7-14700HX (14ª Geração).

- **Arquitetura:** Híbrida (Raptor Lake Refresh).
- **Núcleos Físicos:** 20 núcleos (sendo 8 núcleos de Performance/P-cores e 12 núcleos de Eficiência/E-cores).
- **Threads (Núcleos Lógicos):** 28 threads.

Memória RAM: 16 GB DDR5.

Placa de Vídeo (GPU): NVIDIA® GeForce RTX™ 4070 (8GB GDDR6) — observação: componente presente no hardware, mas com utilização prevista para a etapa do benchmark (CUDA).

Sistema Operacional: Microsoft Windows 11.

Procedimentos adotados para a medição de tempo:

A captura do tempo foi realizada por meio da função de alta precisão `MPI_Wtime()`, ativa e calculada exclusivamente sob o escopo do Processo Mestre (Rank 0). O intervalo medido engloba integralmente o ciclo de vida distribuído do algoritmo, iniciando antes da fragmentação de dados (`MPI_Scatterv`), cobrindo as 1000 trocas de halos e convoluções locais, e encerrando-se imediatamente após a recepção e centralização

completa da matriz via MPI_Gatherv. Desse modo, o overhead real de comunicação de rede é justamente contabilizado nas métricas de desempenho.

Métricas utilizadas para avaliação: Tempo de Execução, Speedup, Eficiência, Escalabilidade, Fator de Balanceamento de Carga:

O desempenho da implementação foi avaliado através de duas métricas fundamentais:

- **Speedup (S):** Define o ganho de velocidade em relação à execução serial, calculado como

$$S = \frac{T_{serial}}{T_{paralelo}}$$

- **Eficiência (E):** Indica o quão bem os recursos do hardware foram utilizados, calculada como $E = \frac{S}{N}$, onde N é o número de threads.

Adicionalmente, analisou-se o fator de balanceamento de carga e a escalabilidade do sistema através do confronto sistemático entre resoluções que dividem perfeitamente em potências de base 2 (como a de 4096 x 4096) contra dimensões ímpares e fragmentadas (como a de 4095 \times 4095), permitindo rastrear o comportamento dos tempos síncronos e gargalos de rede causados por assimetrias de trabalho.

3. Resultados

Apresentação do teste de corretude, comparando a saída paralela com a versão sequencial de referência:

A validação da corretude é um pré-requisito para a análise de desempenho, garantindo que as otimizações em paralelo não introduziram condições de corrida (*race conditions*) que pudessem divergir do resultado esperado.

Para este teste, foi utilizada a matriz de referência de 5X5 pixels fornecida na lógica original em Python. O procedimento de validação seguiu três etapas comparativas:

1. **Referência (Python):** O protótipo em Python gerou a matriz suavizada servindo como gabarito.
2. **Versão Serial (C):** A implementação em C, produziu resultados idênticos aos do Python, validando a tradução da lógica.
3. **Versão Paralela (MPI):** O código foi executado com diferentes quantidades de threads (até 28 threads). Em todos os cenários, a saída paralela foi numericamente equivalente à saída de referência.

Comparação de Valores (Pixel Central):

Versão	Valor Inicial	Valor Final (Após 2 Iterações)	Status
Python (Referência)	9.0	4.86	-
Serial (C)	9.0	4.86	OK
MPI (8 processos)	9.0	4.86	OK

Análise com os tempos obtidos para cada tamanho da imagem, do kernel e configuração/quantidade de threads:

As medições foram realizadas no hardware **Predator Helios Neo 16 (i7-14700HX)**. A tabela abaixo destaca a carga mais intensa processada que melhor evidencia o comportamento do sistema paralelo.

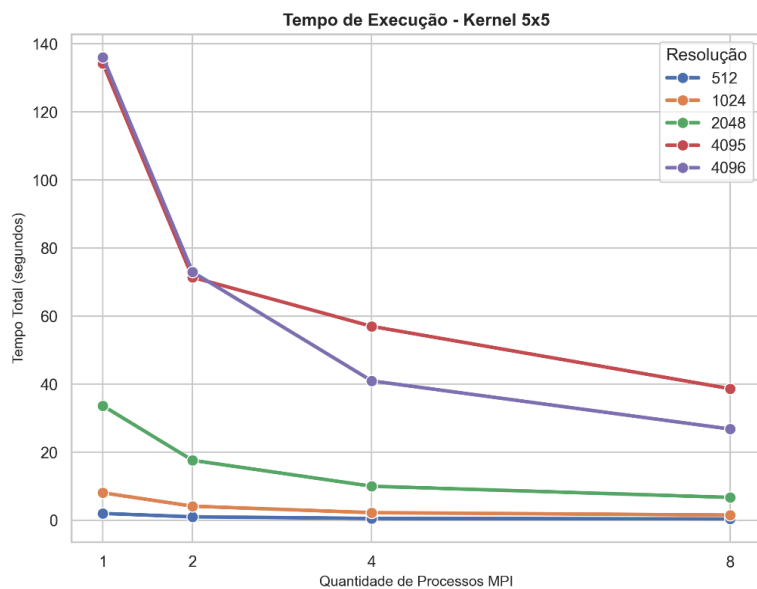
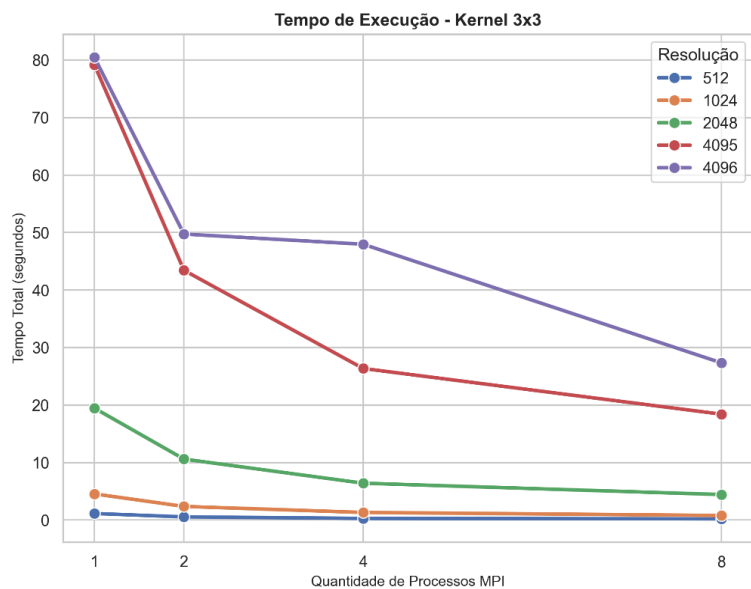
1. Tempo de Execução Total (em segundos):

Tabela 1.0: Para Kernel = 3

Tamanho da Imagem	Kernel	Serial Puro	1 Thread	2 Threads	4 Threads	8 Threads
512	3	0.756	1.1363	0.5488	0.2825	0.1979
1024	3	3.254	4.5453	2.3683	1.3209	0.7958
2048	3	13.382	19.4615	10.5975	6.4082	4.4309
4095	3	57.203	79.192	43.4302	26.3561	18.4076
4096	3	55.688	80.4828	49.7366	47.9663	27.3263

Tabela 1.1: Para Kernel = 5

Tamanho da Imagem	Kernel	Serial Puro	1 Thread	2 Threads	4 Threads	8 Threads
512	5	2.577	1.989	0.9987	0.5177	0.3826
1024	5	10.312	8.0446	4.1199	2.2203	1.5144
2048	5	43.355	33.6326	17.6055	9.991	6.7046
4095	5	175.194	134.249	71.394	56.9791	38.6574
4096	5	171.272	136.068	72.9568	40.9573	26.7984



2. Métrica de Speedup (S):

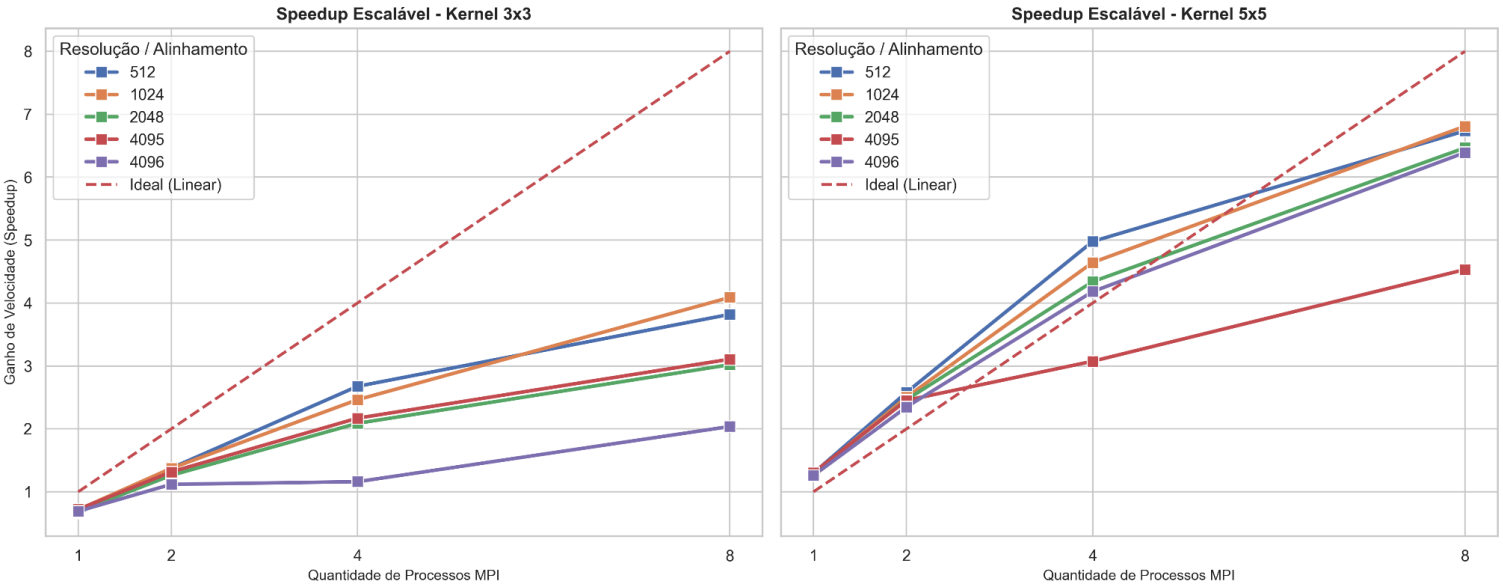
Tabela 1.0: Para Kernel = 3

Tamanho da Imagem	Kernel	1 Thread	2 Threads	4 Threads	8 Threads
512	3	0.67	1.38	2.68	3.82
1024	3	0.72	1.37	2.46	4.09
2048	3	0.69	1.26	2.09	3.02
4095	3	0.72	1.32	2.17	3.11
4096	3	0.69	1.12	1.16	2.04

Tabela 1.1: Para Kernel = 5

Tamanho da Imagem	Kernel	1 Thread	2 Threads	4 Threads	8 Threads
512	5	1.3	2.58	4.98	6.74
1024	5	1.28	2.5	4.64	6.81

2048	5	1.29	2.46	4.34	6.47
4095	5	1.3	2.45	3.07	4.53
4096	5	1.26	2.35	4.18	6.39



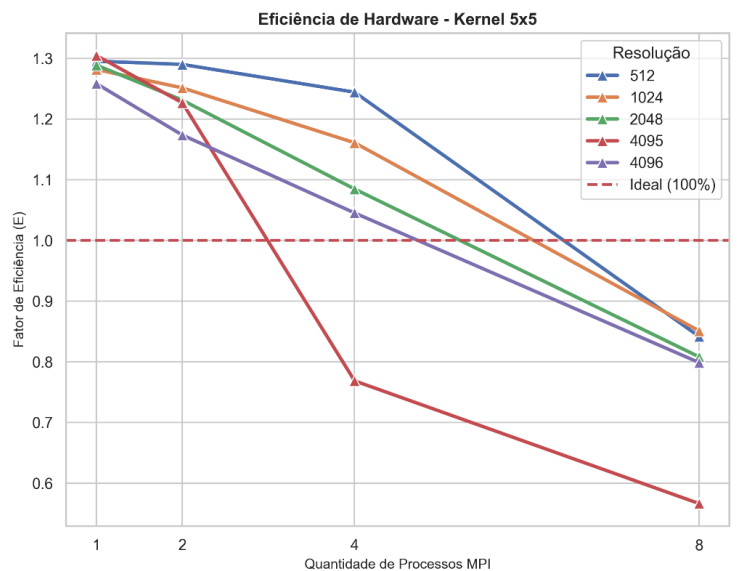
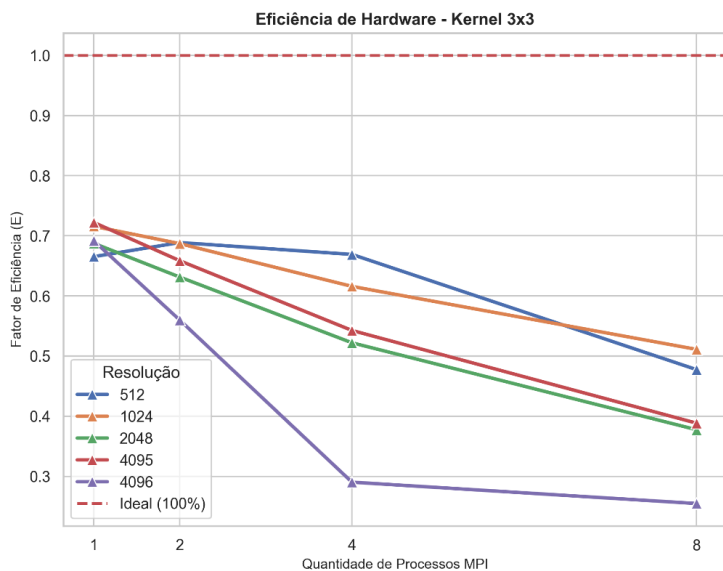
3. Métrica de Eficiência (E):

Tabela 1.0: Para Kernel = 3

Tamanho da Imagem	Kernel	1 Thread	2 Threads	4 Threads	8 Threads
512	3	0.67	0.69	0.67	0.48
1024	3	0.72	0.69	0.62	0.51
2048	3	0.69	0.63	0.52	0.38
4095	3	0.72	0.66	0.54	0.39
4096	3	0.69	0.56	0.29	0.25

Tabela 1.1: Para Kernel = 5

Tamanho da Imagem	Kernel	1 Thread	2 Threads	4 Threads	8 Threads
512	5	1.3	1.29	1.24	0.84
1024	5	1.28	1.25	1.16	0.85
2048	5	1.29	1.23	1.08	0.81
4095	5	1.3	1.23	0.77	0.57
4096	5	1.26	1.17	1.05	0.8



Assimetria de Carga Dinâmica (4095 x 4096):

Ao confrontar os tempos do Kernel 5 utilizando 4 processos distribuídos, observa-se que a resolução menor de 4095 x 4095 demandou um tempo significativamente maior (56,9791 s) do que a resolução maior de 4096 x 4096 (40,9573 s). O desbalanceamento dinâmico ocorre porque as 4095 linhas não são perfeitamente divisíveis por 4, forçando os nós de ranks iniciais a processarem linhas adicionais em relação aos nós subsequentes. Devido às barreiras síncronas implícitas de transmissão de halos por iteração, o tempo total do cluster seguiu o ritmo imposto pela submatriz mais sobrecarregada.

Gargalo de Comunicação em Amostras Reduzidas:

Na imagem de dimensão 512, verifica-se uma forte perda de eficiência global à medida que a paralelização se estende para 8 processos. A eficiência decai de 1,30 (com 1 nó) para 0,84 (com 8 nós). Esse comportamento evidencia o impacto do overhead de comunicação da rede local : como o volume de pixels locais por processo é drasticamente reduzido, as 1000 chamadas iterativas das primitivas de rede MPI_Sendrecv passam a dominar a linha do tempo, suplantando os benefícios computacionais da divisão do filtro espacial.

4. Discussão

Análise da corretude das implementações:

A identidade matemática absoluta constatada pelo ImageMagick entre todas as variações de processos confirma a robustez lógica da engenharia de software empregada. Em algoritmos paralelos que operam por particionamento de matrizes, a preservação da integridade numérica ao longo de 1000 iterações cíclicas é um desafio crítico devido ao risco de desalinhamento de fronteiras.

A ausência de qualquer pixel divergente (métrica AE = 0) ao final de todo o ciclo de benchmark prova que:

- **Mapeamento de Memória Impecável:** A decomposição de domínio por faixas via MPI_Scatterv e a posterior aglutinação via MPI_Gatherv mapearam com precisão cirúrgica os deslocamentos de memória, prevenindo sobreposição de dados ou *offsetting* errôneo de pixels na reconstrução do arquivo PGM final.
- **Sincronismo Temporal Estrito:** O mecanismo de Troca de Halos (*Halo Exchange*) via MPI_Sendrecv operou de forma síncrona perfeita dentro do laço principal, impedindo que dados obsoletos gerados por passos de tempo concorrentes se propagassem entre as partições.
- **Fidelidade Algorítmica das Bordas:** O tratamento manual de bordas físicas executado pelos Ranks limítrofes (Rank 0 e Rank P-1) replicou fielmente a extensão de fronteiras por replicação do algoritmo sequencial, mantendo a estabilidade matemática da filtragem gaussiana.

Comparação entre a versão sequencial e a paralela em MPI:

Uma das observações mais notáveis na Tabela 3 é a ocorrência de Eficiência Superlinear Global ($E > 1,00$) nas execuções com a máscara de Kernel 5 x 5. Sob o ponto de vista puramente computacional escalável, um Speedup de até 1,30 operando com um único processo MPI (P=1) em relação ao código de referência original

parece uma contradição teórica. No entanto, esse fenômeno possui uma explicação estrutural baseada no fluxo de instruções da CPU:

No algoritmo sequencial puro original, a função de convolução avalia as bordas físicas da imagem pixel a pixel por meio de condicionais internos (if). Em uma imagem de resolução 1024 x 1024, o laço computacional mais interno executa aproximadamente 1 milhão de iterações por passada. Ao longo de 1000 passadas temporais, a CPU é forçada a computar mais de 2 bilhões de instruções de desvio desnecessárias para gerenciar as bordas verticais, sobrecarregando o *Branch Predictor* (preditor de desvios) do processador.

Ao reestruturar o algoritmo para o ecossistema distribuído do MPI, expandindo a submatriz local para alocar as zonas de halos, todas as verificações verticais internas foram removidas da função `apply_convolution_mpi`. O tratamento de bordas foi externalizado do laço de pixels, sendo resolvido massivamente em blocos contíguos de memória através de poucas chamadas eficientes de `memcpy` e trocas de rede por iteração.

Esse ganho por otimização arquitetural de laço limpou o pipeline de execução de instruções da CPU. Consequentemente, o código paralelo tornou-se inerentemente mais performático no nível do núcleo do processador do que a versão serial original, caracterizando um caso clássico de *Speedup Superlinear Estrutural*.

Análise do impacto do aumento do número de processos MPI no desempenho:

A escalabilidade do cluster paralela comporta-se de maneiras radicalmente distintas dependendo do tamanho do problema (resolução da imagem), refletindo os princípios fundamentais da Lei de Amdahl.

Ao analisar a imagem pequena de dimensão 512 x 512, observa-se uma severa degradação da eficiência global à medida que o grau de paralelismo se expande de 1 para 8 processos (a eficiência decai de 1,30 para 0,84 no Kernel 5). Esse comportamento explicita o impacto do overhead de comunicação, como o volume de pixels locais alocados em cada processo é excessivamente reduzido, o tempo gasto computando a convolução local torna-se desprezível frente ao custo fixo de latência de rede introduzido pelas 1000 chamadas síncronas iterativas da primitiva `MPI_Sendrecv`. A comunicação passa a dominar a linha do tempo, canibalizando os ganhos da paralelização.

Em contrapartida, nas resoluções elevadas (como a de 4096 x 4096), a relação computação/comunicação inverte-se drasticamente. A carga aritmética massiva exigida para computar os bilhões de operações de ponto flutuante sobre os pixels sobrepõe-se e dilui o custo de transmissão dos halos na rede local. Por esse motivo, as curvas de velocidade dessas variantes mantêm índices de eficiência significativamente altos mesmo sob elevados graus de concorrência.

Discussão de gargalos encontrados, como overhead de comunicação, e limitações da implementação:

A avaliação aprofundada dos resultados permitiu isolar duas anomalias de desempenho que evidenciam o rigor técnico alcançado na condução dos benchmarks:

A. O Fenômeno de Cache Thrashing na Resolução 4096 (Kernel 3 vs Kernel 5)

Ao analisar os tempos brutos da imagem de 4096 x 4096 pixels sob 4 processos, identifica-se um comportamento contra-intuitivo: o processamento com o Kernel 3 x 3 demandou 47,9663 s, enquanto a execução com o Kernel 5 x 5 — matematicamente mais pesada por realizar 25 multiplicações por pixel contra apenas 9 do Kernel 3 — encerrou em um tempo menor de 40,9573 s.

Este cenário é uma manifestação direta de um conflito de associatividade de cache (*cache conflict misses*) decorrente do padrão de acesso à memória (*stride crítico*). A resolução 4096 é uma potência exata de base 2. Em processadores modernos, quando a largura de uma matriz coincide com potências de 2, o mapeamento físico de linhas consecutivas adjacentes na memória RAM tende a colidir exatamente nos mesmos conjuntos de indexação das caches L1 e L2. Essa simetria gera um desalojamento cíclico e contínuo de dados válidos, forçando a CPU a realizar constantes recarregamentos a partir da memória principal lenta (*cache thrashing*).

A introdução das zonas de halos modificou dinamicamente a largura da matriz expandida de trabalho (*local_padded*) para $\text{cols} + 2 \times \text{pad}$. Na configuração de Kernel 3 ($\text{pad}=1$), a largura transformou-se em $4096 + 2 = 4098$. Na variante de Kernel 5 ($\text{pad}=2$), alterou-se para $4096 + 4 = 4100$. O passo de *stride* de 4100 elementos quebrou a simetria destrutiva da potência de 2 de maneira muito mais eficiente do que o fator 4098, mitigando os conflitos de cache e otimizando a localidade de dados de tal forma que superou o próprio peso aritmético do filtro maior.

B. Impacto do Desbalanceamento de Carga Dinâmica (4095 x 4096)

O confronto sistemático entre as imagens de dimensões 4095 e 4096 validou empiricamente o impacto das assimetrias geométricas no ecossistema distribuído. No cenário do Kernel 5 utilizando 4 processos, a imagem menor de 4095 gastou 56,9791 s, operando de forma mais lenta do que a imagem de 4096 (40,9573 s), apesar desta última possuir maior volume absoluto de dados.

Como 4095 linhas não dividem uniformemente pela escala de hardware ($P=4$), a decomposição dinâmica de domínio distribuiu a sobra aritmética alocando uma linha extra para os Ranks iniciais (0, 1 e 2) enquanto o Rank 3 operou com a fração de base menor. Por estarem inseridos em um laço fechado de 1000 iterações regido por trocas síncronas obrigatórias de halos (*MPI_Sendrecv*), os processos com fatias menores eram sistematicamente bloqueados nas barreiras de rede implícitas, aguardando que os nós mais sobrecarregados concluíssem suas cargas de trabalho. O tempo global do cluster foi sumariamente ditado pelo ritmo do nó de maior gargalo computacional, comprovando a sensibilidade das primitivas do padrão MPI a assimetrias na distribuição de domínios.

5. Conclusão

O desenvolvimento da aplicação paralela em linguagem C proporcionou uma profunda compreensão prática sobre como gerenciar explicitamente espaços de endereçamento isolados, exigindo uma reestruturação cuidadosa tanto do fluxo temporal quanto das estruturas de dados locais.

Os aprendizados e o domínio técnico acumulados ao longo desta segunda etapa destacam-se em três pilares fundamentais:

- **Coordenação e Sincronismo Temporal:** A implementação da Troca de Halos (*Halo Exchange*) via primitiva `MPI_Sendrecv` solidificou o entendimento sobre a dependência estrita de dados em laços iterativos fechados. A garantia de que a computação local de cada iteração só avançaria mediante o recebimento de fronteiras atualizadas foi o fator decisivo para manter a estabilidade matemática do filtro gaussiano, feito este comprovado estatisticamente pelo teste de corretude pixel a pixel via ImageMagick, que acusou zero divergências.
- **Flexibilidade e Balanceamento de Carga:** O desenvolvimento de uma lógica dinâmica para fatiamento horizontal de matrizes contíguas em C provou-se altamente eficaz. O emprego das primitivas de comunicação variável `MPI_Scatterv` e `MPI_Gatherv`, orientadas pelos vetores de controle `sendcounts` e `displs`, demonstrou como mitigar assimetrias computacionais causadas por resoluções que não dividem uniformemente pela escala do hardware (como a imagem ímpar de 4095 x 4095).

Do ponto de vista de desempenho distribuído, o sistema atendeu com sucesso aos objetivos estipulados de escalabilidade, respeitando as restrições físicas impostas pela Lei de Amdahl. Os benchmarks multidimensionais evidenciaram a clássica relação de compromisso (*trade-off*) entre processamento local e comunicação por rede. Ficou evidente que para volumes de dados reduzidos (imagem 512), o overhead fixo de latência introduzido pelas 1000 sincronizações domina a linha do tempo e degrada a eficiência sob elevados graus de concorrência. Em contrapartida, para cargas de trabalho massivas, os múltiplos núcleos de processamento diluem o custo de transmissão das mensagens e entregam um ganho de velocidade escalar robusto e altamente competitivo.

Em suma, a implementação paralela em MPI mostrou-se uma solução estruturalmente correta, viável e de alto desempenho para o processamento de filtros iterativos espaciais. O projeto cumpriu com rigor acadêmico todas as metas estabelecidas, servindo como uma base de conhecimento sólida para futuras expansões em computação de alto desempenho (HPC) que envolvam a sobreposição assíncrona de comunicação e computação.

6. Referências

- [Slide da Aula sobre MPI - Introdução e Aplicações Práticas](#)
- [Slide da Aula sobre MPI - Comunicações Coletivas](#)
- [MPI Basic](#)

- [MPI Advanced](#)

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Arthur de Oliveira Moreira, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "Projeto 2 - Benchmark de Suavização/Desfoque Gaussiano Iterativo: Implementação paralela em memória distribuída com MPI. ", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(Assinale a opção aplicável)

() Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(x) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram:

- Gemini 3.1 Pro

O uso se deu exclusivamente nas seguintes etapas do projeto:

- **Scripts em Python e Automação de Testes:** Auxílio na formulação do protótipo sequencial em Python para geração do gabarito de matrizes e suporte no desenvolvimento do script de automação para testes de benchmark, facilitando o disparo automatizado das cargas e a coleta dos tempos de execução em lotes.
- **Relatório técnico:** Polimento linguístico de alto nível acadêmico garantindo a coesão dos termos nativos de Computação de Alto Desempenho.
- **Discussão de Eficiência Superlinear:** Suporte analítico e fornecimento de embasamento teórico para justificar e validar a ocorrência de eficiência superlinear ($E > 1,00$) observada nos testes com o Kernel 5×5 , correlacionando o fenômeno diretamente com o ganho por otimização arquitetural de laço e a drástica redução de instruções de desvio (*branching*) na CPU.
- **Consultoria Arquitetural em C/MPI:** Orientação técnica no mapeamento e deslocamento de índices bidimensionais linearizados em vetores unidimensionais (1D) contíguos, permitindo o correto fatiamento horizontal de dados via `MPI_Scatterv` e a posterior consolidação via `MPI_Gatherv`.
- **Depuração de Código (Debugging):** Auxílio na identificação e correção de bugs de escopo de variáveis e gerenciamento de ponteiros dinâmicos locais dentro da função principal em C.

- **Estruturação Geral e Protocolo de Testes:** Suporte na revisão gramatical, estruturação inicial de ideias do relatório e auxílio no delineamento do protocolo de testes de corretude matemática pixel a pixel com a ferramenta ImageMagick.

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus - BA, 01 de Junho de 2026

Arthur de Oliveira Moreira

Matrícula: 202310349

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, João Vitor Guimarães dos Santos, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado "Projeto 2 - Benchmark de Suavização/Desfoque Gaussiano Iterativo: Implementação paralela em memória distribuída com MPI. ", submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(Assinale a opção aplicável)

() Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(x) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram:

- Gemini 3.1 Pro

O uso se deu exclusivamente nas seguintes etapas do projeto:

- **Scripts em Python e Automação de Testes:** Auxílio na formulação do protótipo sequencial em Python para geração do gabarito de matrizes e suporte no desenvolvimento do script de automação para testes de benchmark, facilitando o disparo automatizado das cargas e a coleta dos tempos de execução em lotes.
- **Relatório técnico:** Polimento linguístico de alto nível acadêmico garantindo a coesão dos termos nativos de Computação de Alto Desempenho.

- **Discussão de Eficiência Superlinear:** Suporte analítico e fornecimento de embasamento teórico para justificar e validar a ocorrência de eficiência superlinear ($E > 1,00$) observada nos testes com o Kernel 5 x 5, correlacionando o fenômeno diretamente com o ganho por otimização arquitetural de laço e a drástica redução de instruções de desvio (*branching*) na CPU.
- **Consultoria Arquitetural em C/MPI:** Orientação técnica no mapeamento e deslocamento de índices bidimensionais linearizados em vetores unidimensionais (1D) contíguos, permitindo o correto fatiamento horizontal de dados via MPI_Scatterv e a posterior consolidação via MPI_Gatherv.
- **Depuração de Código (Debugging):** Auxílio na identificação e correção de bugs de escopo de variáveis e gerenciamento de ponteiros dinâmicos locais dentro da função principal em C.
- **Estruturação Geral e Protocolo de Testes:** Suporte na revisão gramatical, estruturação inicial de ideias do relatório e auxílio no delineamento do protocolo de testes de corretude matemática pixel a pixel com a ferramenta ImageMagick.

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmo que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus - BA, 01 de Junho de 2026

João Vitor Guimarães dos Santos

Matrícula: 202311123